Disentangling Information Sharing in Multi-Metric Runtime Forecasting for Cloud Software Systems

Abdelhak Kadouma[a,*], Amit Shukla[a] and Mohammed Elmusrati[a]

[a] University of Vaasa, Vaasa, Finland

ARTICLE INFO

Keywords:
runtime forecasting
cloud telemetry
multi-metric forecasting
multi-task learning
negative transfer
shared representations

ABSTRACT

Multi-metric runtime forecasting involves two distinct choices: using histories from other resources and sharing a predictive representation across resource targets. We compare S1 (own-resource input, independent target model), S2 (all-resource input, independent target model) and M1 (all-resource input, shared multi-resource model). Operational contrasts evaluate contextual-input value (S2 versus S1) and implemented representation sharing (M1 versus S2). A conventional long short-term memory (LSTM) design is examined through a TimeTrack development study, Bitbrains fastStorage and Rnd external environments, and Materna T1–T3 dependent temporal replication. The resources are central processing unit (CPU) utilisation, throughput, memory usage and disk-write activity. Cross-resource context is selective, varying by resource, environment and time. Memory shows the clearest repeated contextual benefit, but not in every block. M1 versus S2 is adverse for throughput, memory and disk write in all six evidence blocks; CPU has limited positive exceptions. M1 generally underperforms S1. In the five-horizon external instantiation, M1 uses approximately 74.5% fewer parameters and 75% fewer models than S2, establishing an accuracy–efficiency boundary. For the evaluated resources, eligible populations and conventional shared-LSTM formulation, additional telemetry and predictive representation sharing should be evaluated as separate design decisions.

1. Introduction

Cloud software systems expose multiple streams of runtime telemetry. Monitoring records past and present resource use; forecasting can supply advance information for adaptation and resource management (Taherizadeh, Jones, Taylor, Zhao and Stankovski, 2018; Bauer, Züfle, Herbst, Zehe, Hotho and Kounev, 2020). For software engineers, choosing a forecaster therefore involves deciding which observations it should use and how its predictions should be organised, not merely selecting an algorithm. Prediction-based provisioning and autoscaling research also distinguish forecast quality from the consequences of acting on a forecast (Calheiros, Masoumi, Ranjan and Buyya, 2015; Qu, Calheiros and Buyya, 2018). Better predictions may inform proactive decisions, but their operational value depends on decision policies and system constraints. We examine the forecast-design choices supplying such systems, without deploying an autoscaler or claiming improved SLA compliance, reduced cloud cost, energy savings or failure prevention.

Established forecasting research already covers several ways of learning from runtime and resource telemetry. Independent runtime-series benchmarking spans classical statistical, neural and zero-shot foundation models (Di Menna, Traini and Cortellessa, 2026). Cloud studies include multivariate resource inputs, recurrent architectures, joint multi-output prediction and hybrid models evaluated across multiple provider environments (Ullah, Bilal and Yoon, 2023; Tripathi, Priyadarshni, Misra and Singh, 2025). Global forecasting also learns parameters across collections of series (Salinas, Flunkert, Gasthaus and Januschowski, 2020). These capabilities are established. We ask how information sharing affects particular resource targets, not whether multi-resource prediction is possible. More inputs, joint outputs and fewer models require separate evaluation; our contribution is experimental evidence, not another recurrent architecture.

We distinguish *input information sharing* from *predictive representation sharing*: a target-specific model may use other resources' histories without sharing learned parameters across targets. A common representation instead learns several targets jointly. Multi-task learning offers inductive transfer through related tasks (Caruana, 1997), but task combinations and sharing arrangements require evaluation (Standley, Zamir, Chen, Guibas, Malik and Savarese, 2020; Fifty, Amid, Zhao, Yu, Anil and Finn, 2021; Nochumsohn, Zisling and Azencot, 2025). Independent all-input target models therefore provide an intermediate control, making contextual-input and implemented sharing outcomes separately observable without presuming that either improves accuracy.

The closest examined designs span shared ten-channel prediction (Ullah et al., 2023), independently forecast runtime series and zero-shot foundation models (Di Menna et al., 2026), and MMRP's univariate/multivariate single-target CPU and memory forecasting (Tripathi et al., 2025). MMRP is meaningful S1–S2-related prior art, not a comparison introduced by our study. Among these closest examined designs, none reports the complete within-study contrast between own-resource forecasting, independently trained target models receiving cross-resource histories, and joint resource forecasting through a shared representation. The independent-to-shared all-input bridge is the important

*Corresponding author

abdelhak.kadouma@uwasa.fi (A. Kadouma); amit.shukla@uwasa.fi (A. Shukla); moel@uwasa.fi (M. Elmusrati)

missing operational comparison. This bounded finding concerns these reported designs, not an absence throughout the literature.

We evaluate conventional LSTM formulations with independent target models using own-resource histories (S1) or all-resource histories (S2), and a shared representation with resource-specific heads using all-resource histories (M1). S2 versus S1 evaluates contextual information, M1 versus S2 the implemented sharing transition, and M1 versus S1 the complete joint system. These are matched operational contrasts, not pure causal interventions: capacities and selected settings differ, and the arms are not a complete 2×2 factorial. The evaluated resources are CPU utilisation, throughput, memory usage and disk-write activity. TimeTrack supplies development evidence, Bitbrains fastStorage and Rnd provide external environments, and Materna T1–T3 provide dependent temporal replication. Eligible-population restrictions delimit applicability. Sections 3 and 4 define the formulations and evaluation; persistence also distinguishes formulation-relative gains from absolute forecasting skill.

The evidence in Section 6 identifies selective contextual value: S2 versus S1 varies with resource, environment and temporal block. Memory has the clearest repeated positive pattern, but not universally. For M1 versus S2, throughput, memory and disk write have negative primary gains in all six evidence blocks; CPU has limited positive exceptions. M1 versus S1 is negative in every displayed resource/block combination except Materna T2 CPU. In the five-horizon external instantiation, M1 nevertheless uses approximately 74.5% fewer parameters and 75% fewer models than S2. Consolidation therefore accompanies predictive degradation within the evaluated shared-LSTM design, rather than establishing a general verdict on multi-task forecasting. We make four contributions:

1. We formulate explicit matched S1/S2/M1 operational contrasts, including shared versus independent all-input prediction, while acknowledging existing univariate/contextual comparisons and unequal capacity and selection settings.
2. We provide evidence on how contextual-input value varies by resource, environment and time across development, external environments and dependent temporal replication within eligible populations.
3. We provide replicated adverse shared-representation evidence for the evaluated conventional shared-LSTM implementation relative to independent all-input target models, without claiming discovery of negative transfer or identifying its mechanism.
4. We jointly characterise model/parameter consolidation, observed fitting workload, predictive accuracy and applicability boundaries for the evaluated implementation, without extrapolating to unmeasured production savings or excluded populations.

2. Related Work

2.1. Runtime and Cloud-Resource Forecasting

Runtime monitoring supplies observations about software behaviour and the resources supporting it. In self-adaptive and cloud systems, these observations inform decisions about configuration, capacity and resource allocation (Taherizadeh et al., 2018; Bauer et al., 2020). Forecasting extends this information beyond the current state, potentially allowing adaptation before demand changes take effect. However, predictive accuracy is only one component of a management loop: allocation policies, actuation delays and uncertainty also affect operational outcomes (Qu et al., 2018). Forecast quality and controller effectiveness therefore require distinct evaluation.

This distinction is reflected in prediction-based provisioning and software-service research. ARIMA workload prediction has been investigated together with its consequences for cloud application quality of service, whereas empirical service-quality forecasting compares the accuracy and cost of prediction methods (Calheiros et al., 2015; Syu, Kuo and Fanjiang, 2017). These studies establish forecasting as a software-engineering concern rather than solely an algorithmic benchmark. They do not imply that a lower forecasting error automatically improves service levels. The present study evaluates telemetry forecasts, not an autoscaling controller, service-level compliance or deployed resource-allocation policy.

Recent runtime forecasting also extends beyond infrastructure utilisation. Di Menna et al. (2026) compare classical statistical, neural and foundation models on 110 daily runtime series from 29 applications, including crash rates and several application-time measurements. Forecasts extend to 14 days. The experiments operate independently on each metric: conventional models are fitted per series, while foundation models are used zero-shot on individual histories. Multivariate forecasting is left for future work. This model-family comparison informs runtime prediction, but does not evaluate sharing among jointly predicted resource tasks.

Cloud-resource forecasting already includes correlated multivariate inputs, deep recurrent predictors and joint resource outputs (Tran, Nguyen, Nguyen and Nguyen, 2018; Xu, Song, Wu, Gill, Ye and Xu, 2022; Ullah et al., 2023). These capabilities establish the resource-forecasting problem addressed here, rather than an architectural contribution of the present study.

Hybrid architectures, uncertainty-aware predictions and evaluations on multiple cloud providers further broaden this evidence (Tripathi et al., 2025; Rossi, Visentin, Carraro, Prestwich and Brown, 2025; Zhang, Yang, Yu and Pan, 2025). Online multivariate resource forecasting and microservice-indicator forecasting address additional operational settings (Gao, Tang, Yao, Wu, Yuan, Zhou, Li and Chen, 2026b; Gao, Yu, Luo, Ma, Sun, Fang and Gao, 2026a). Thus multivariate inputs, recurrent resource models, joint forecasting and multi-provider evaluation are established capabilities, not architectural contributions of this study.

The TimeTrack dataset places these questions in an OAI CI/CD setting. Meliani, Arora, Ksentini and Knopp (2025) document its computing and network telemetry and report contemporaneous cross-resource associations, including CPU–memory relationships on some machines. Such associations motivate considering other resource histories, but do not establish incremental predictive value after conditioning on the target's own history. That distinction requires a forecasting comparison rather than an inference from correlation alone.

2.2. Information Scope and Sharing in Forecasting

Three design choices must be distinguished: which channels supply historical inputs, which variables form the prediction targets, and which learned parameters are common to those targets. A multichannel input can support a single target; multiple future values can instead represent different horizons of that target. Neither arrangement by itself establishes resource-task sharing. Conversely, sharing parameters does not require mixing channels in each input window.

The closest studies instantiate different parts of this design space (Table 1). In particular, MMRP's univariate/multivariate single-target comparisons (Tripathi et al., 2025) are a meaningful contextual-input precedent related to S1–S2, not evidence of cross-target sharing.

Other designs reinforce the distinction between covariates and targets. The esDNN study forecasts CPU usage from several resource covariates. Models that account for uncertainty instead compare predictions for one resource with joint processing-unit and memory forecasts, alongside transfer learning (Xu et al., 2022; Rossi et al., 2025). These are different ways of using several resource series, rather than interchangeable definitions of multivariate forecasting.

The distinction also matters outside cloud telemetry. PatchTST processes channels independently while reusing embedding and Transformer weights across them (Nie, Nguyen, Sinthong and Kalagnanam, 2023). Channel independence therefore does not mean independently fitted models. In contrast, iTransformer represents individual variates as tokens and models relationships between them through attention (Liu, Hu, Zhang, Wu, Wang, Ma and Long, 2024). The relevant difference here is how information is combined, not a general comparison of Transformer architectures.

Sharing can also occur across series. DeepAR learns recurrent parameters across collections of related series, while the local/global forecasting framework distinguishes separately fitted functions from a common function learned across series (Salinas et al., 2020; Montero-Manso and Hyndman, 2021). Such sharing need not provide every series as simultaneous input channels. Together, these designs show why input availability, output organisation and shared predictive representation require separate experimental controls.

2.3. Multi-Task Learning, Task Relatedness, and Adverse Transfer

Multi-task learning uses relationships among prediction tasks to provide an inductive bias through shared learning (Caruana, 1997). Common representations can transfer useful information, but the benefit depends on task relationships, data and the chosen sharing structure (Yu, Liu, Luo, Huang, Zhou, Liu, Hu, Chen, Zhang, Zhang, Shen, Adhikarla, Dai, Zhang, Kong, Ye, Yin, Namboodiri, Davison, Moore and Chen, 2025). Learning all available tasks together is therefore not an unconditional improvement over separate models. Task-grouping research makes this dependence explicit by selecting combinations under accuracy and computational constraints, or by estimating how learning one task affects another (Standley et al., 2020; Fifty et al., 2021).

Optimisation research addresses a related but distinct issue: joint objectives can favour different updates. Multi-objective formulations and gradient modification methods provide alternatives to an unexamined fixed combination of task losses (Sener and Koltun, 2018; Yu, Kumar, Gupta, Levine, Hausman and Finn, 2020). These results motivate testing sharing rather than assuming that nominally related outputs should always use one representation. They do not establish that gradient conflict, task interference or competition for capacity occurs in a particular resource forecaster without corresponding measurements.

Forecasting-specific multi-task work already investigates such questions. NARMAX-LSTM shares LSTM layers across station-level forecasting tasks, compares single-task baselines and explicitly discusses negative transfer (Nikentari and Wei, 2022). MTLinear treats variates or groups as tasks, analyses gradient relationships and balance, and evaluates grouping and efficiency (Nochumsohn et al., 2025). Cloud workload forecasting has also been formulated through multi-task learning, with a task construction different from our resource-head formulation (Bi, Yuan, Zhou and Liu, 2019). Negative transfer and selective sharing are thus established concepts. For the present comparison, degradation after introducing M1 relative to independent all-input S2 models is operationally consistent with adverse transfer; it does not by itself identify an optimisation or capacity mechanism. Neither task grouping nor gradient modification is evaluated in the present experiment.

2.4. Evaluation Gap and Study Scope

The closest inspected designs cover individual parts of the information-sharing design space, but not the complete within-study S1/S2/M1 operational comparison (Table 1). This bounded observation is not a literature-wide absence or priority claim.

S2 serves as the intermediate experimental control, rather than introducing multivariate single-target forecasting. It allows the value of additional resource histories to be examined separately from the implemented transition to cross-target representation sharing. S1–S2 and S2–M1 are matched operational contrasts under common resource,

Table 1
Bounded comparison of the closest inspected studies. Organisational analogues do not imply identical architectures; codes are defined below. No priority claim is implied.

Study	Input / target organisation	Reported S1/S2/M1 analogues	S1–S2 contextual comparison	S2–M1 sharing comparison	Scope relative to this study
Ullah et al. (2023)	Ten-channel histories to joint ten-channel forecasts through a common multivariate BiLSTM.	S1: NO S2: NO M1: PARTIAL	NO	NO	fastStorage/Rnd joint forecasting; no independent all-input target bridge. Common dense output differs from our resource-head topology.
Di Menna et al. (2026)	Per-series classical/neural fits; foundation models forecast individual runtime histories zero-shot.	S1: YES S2: NO M1: NO	NO	NO	Independent-series/model-family evaluation, not multivariate runtime forecasting. Pretrained weight reuse is not M1.
Tripathi et al. (2025) (MMRP)	Univariate or multivariate histories to one target series; CPU/memory evaluated separately.	S1: YES S2: PARTIAL M1: NO	PARTIAL	NO	Alibaba/Google contextual comparison; no reported joint resource-sharing arm. Feature/independent-fit matching is partially specified.
This study	S1: own-resource history, independent target model. S2: all-resource histories, independent target model. M1: all-resource histories, shared representation and resource-specific outputs.	S1: YES S2: YES M1: YES	YES	YES	Development, external environments and dependent temporal replication; both operational bridges, not parameter-matched causal interventions or complete factorial isolation.

YES: explicitly represented in the inspected reported experiment. NO: not part of the inspected reported experiment; this does not assert that an unpublished or technically possible implementation does not exist. PARTIAL: related but non-equivalent or incompletely specified.

horizon and temporal support, not pure causal interventions. S1/S2 differ in input width, parameter count and selected settings; S2/M1 also differ in aggregate capacity, model count, optimisation and target-specific versus common context selection. The decomposition therefore characterises the evaluated implementations, not a parameter-matched or complete factorial effect.

The study retains these contrasts through TimeTrack development evidence, separate Bitbrains fastStorage and Rnd external environments, and Materna T1–T3 as dependent temporal replication. Explicit eligible-population restrictions delimit applicability, while forecasting accuracy and model-consolidation evidence remain separate outcomes. This scope supports resource-, environment- and time-dependent assessment without treating consecutive blocks as independent datasets or extending conclusions to all shared architectures. Section 3 defines the formulations and the matched comparisons used to address this question.

3. Multi-Metric Runtime Forecasting Framework

The framework distinguishes access to resource histories from sharing a learned predictive representation across resource targets. Figure 1 summarises S1, S2 and M1 and their matched operational contrasts.

3.1. Problem Formulation

Let $\mathcal{R} = \{\text{cpu}, \text{thr}, \text{mem}, \text{disk}\}$ denote the resource set and let $R = |\mathcal{R}| = 4$. At discrete time t, the multivariate telemetry observation is $\mathbf{x}_t = (x_t^{(r)})_{r\in\mathcal{R}} \in \mathbb{R}^R$. For a context length $L \in \mathbb{N}$, the complete history available at forecasting origin t is

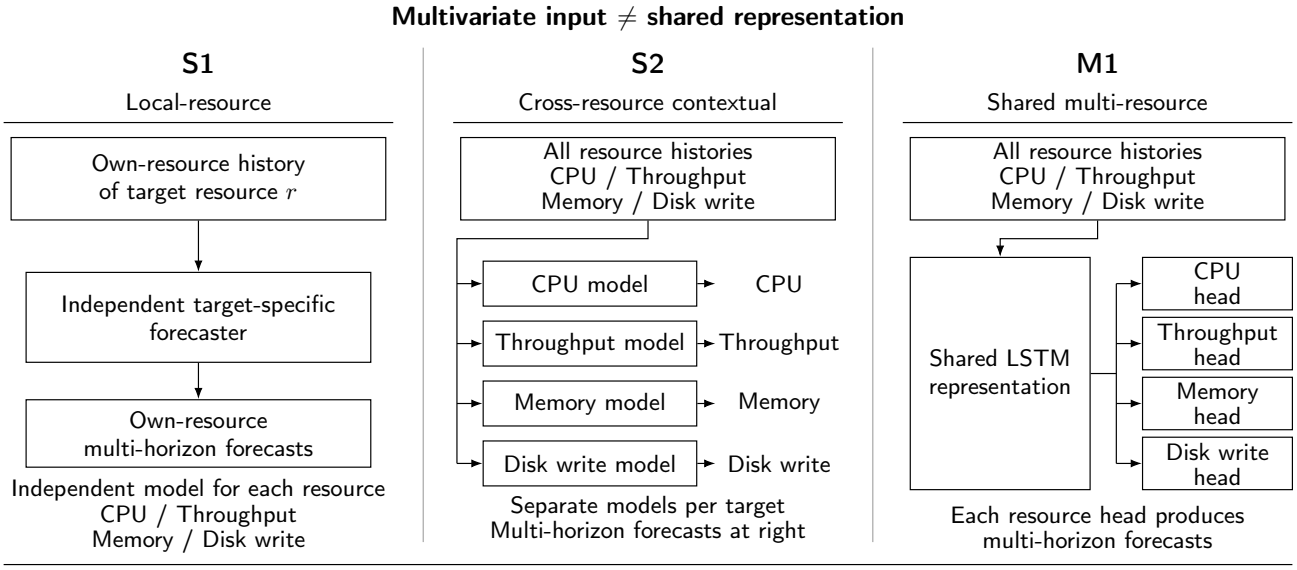
$$\mathbf{X}_{t,L} = [\mathbf{x}_{t-L+1}, \dots, \mathbf{x}_t]^\top \in \mathbb{R}^{L\times R}. \tag{1}$$

Let $\mathcal{H} \subset \mathbb{N}_{>0}$ be a finite set of forecast horizons. For target resource r, $y_{t+h}^{(r)}$ is the future target at horizon $h \in \mathcal{H}$ and $\mathbf{y}_{t,\mathcal{H}}^{(r)} = (y_{t+h}^{(r)})_{h\in\mathcal{H}}$ is its multi-horizon target vector. A forecaster produces

$$\hat{\mathbf{y}}_{t,\mathcal{H}}^{(r)} = F_r(\mathcal{I}_{t,L}; \boldsymbol{\theta}), \tag{2}$$

where $\mathcal{I}_{t,L}$ denotes the formulation-dependent information set and $\boldsymbol{\theta}$ denotes the relevant model parameters.

The context contains only observations available at or before t, whereas each target lies strictly after the forecasting origin. Resource channels are aligned within a context, and the horizon index is interpreted on the time grid of the dataset under study. This notation does not require one universal sampling interval: elapsed-time mappings can differ across datasets provided that the ordering of observations, origins, and future targets is preserved. An eligible origin must supply the information required by every formulation and the complete target vector used in their comparison.



S2 versus S1	Cross-resource contextual information
M1 versus S2	Implemented representation sharing
M1 versus S1	Complete joint-system change

Figure 1: S1 fits independent target models from own-resource histories; S2 supplies all histories to each independently trained target model. M1 uses all histories with a shared LSTM representation and resource-specific heads. S2 versus S1 assesses contextual input; M1 versus S2 the implemented move to representation sharing; M1 versus S1 the complete joint-system change. These are matched operational contrasts, not a complete 2×2 factorial, parameter-matched interventions or pure causal isolation of either design choice.

Comparisons among the three formulations use matched forecasting origins, targets, horizons, and data partitions. Consequently, the contrasts concern changes in information and parameter sharing rather than changes in the evaluated sample.

3.2. Resource Metrics and Multi-Horizon Forecasting

The resource vector comprises CPU utilisation, aggregate throughput, memory usage, and disk-write activity. CPU utilisation represents processor demand; throughput represents the aggregate rate of data transferred through the runtime environment; memory usage represents occupied memory; and disk-write activity represents write traffic to persistent storage. These channel definitions are intentionally independent of dataset-specific field names, units, and sampling intervals. Their concrete mappings are specified with the experimental data in Section 4.

Every forecasting origin yields one prediction for each $h \in \mathcal{H}$. The horizon set is therefore a property of an experimental instantiation rather than a restriction of the framework: $\mathcal{H}$ may be any finite set of positive offsets supported by the data. The same distinction applies to L. Candidate context lengths and horizon values are fixed before model evaluation, but their numerical values may vary with sampling cadence without changing the S1, S2, or M1 definitions. Each formulation predicts its full horizon vector jointly, rather than fitting a separate model for every horizon.

The four channels have distinct operational meanings and are not treated as interchangeable measurements. A resource may nevertheless carry contemporaneous or lagged information relevant to another target. The framework makes this a testable modelling question by varying access to the channel histories while holding the target definition fixed. Joint horizon prediction similarly holds the output organisation fixed across S1, S2, and M1, so that information sharing is not confounded with a change from single- to multiple-horizon training.

Resource channels may differ substantially in scale and distribution. Model training therefore operates on resource-specific transformed and scaled targets, while evaluation can return forecasts to their native resource scales. The transformations and training-partition scaling rules belong to the experimental instantiation. At the framework level, their purpose is to place resource errors on comparable optimisation scales, particularly when a single objective combines several target resources.

3.3. S1 Local-Resource Forecasting

S1 is the local-information formulation. For target resource r, its input is the single-channel sequence

$$\mathbf{X}_{t,L}^{\mathrm{S1},r} = [x_{t-L+1}^{(r)}, \dots, x_t^{(r)}]^\top \in \mathbb{R}^{L\times 1}, \tag{3}$$

and its output is

$$\hat{\mathbf{y}}_{t,\mathcal{H}}^{(r)} = f_r^{\mathrm{S1}}(\mathbf{X}_{t,L}^{\mathrm{S1},r}; \boldsymbol{\theta}_r^{\mathrm{S1}}) \in \mathbb{R}^{|\mathcal{H}|}. \tag{4}$$

A distinct parameter set θ_r^{S1} is learned for each resource; no learned representation is shared across targets. Each S1 model has its own temporal encoder, prediction mapping and optimisation objective for the selected target resource and horizons. S1 remains multi-horizon and provides the local-history baseline. Using the same model family across formulations focuses the comparison on information structure. S1 withholds non-target channels; it does not assume that resources are independent.

3.4. S2 Cross-Resource Contextual Forecasting

S2 expands the information set while retaining target-specific modelling. For every $r \in \mathcal{R}$, the input is the complete history

$$\mathbf{X}_{t,L}^{S2,r} = \mathbf{X}_{t,L} \in \mathbb{R}^{L\times R}, \qquad \hat{\mathbf{y}}_{t,\mathcal{H}}^{(r)} = f_r^{S2}(\mathbf{X}_{t,L}; \theta_r^{S2}). \quad (5)$$

Each θ_r^{S2} is trained independently. Non-target histories act as covariates, but the output and loss remain specific to r: gradients from one target cannot update another model's representation. Thus all-resource input does not establish shared learning across targets.

Under matched support, S2 versus S1 is a controlled contrast between all-resource and own-resource histories while preserving the separate-model organisation. It isolates the empirical contribution of cross-resource context under the specified design; it does not presume that every additional channel is informative, nor that the contribution has a particular sign.

3.5. M1 Shared Multi-Resource Forecasting

M1 combines all-resource input with a representation shared across target tasks. A temporal encoder E_θ maps the complete context to a latent vector $\mathbf{z}_t$, and one resource-specific head g_{r,ϕ_r} maps that representation to the horizon vector for resource r:

$$\begin{aligned} \mathbf{z}_t &= E_\theta(\mathbf{X}_{t,L}), \\ \hat{\mathbf{y}}_{t,\mathcal{H}}^{(r)} &= g_{r,\phi_r}(\mathbf{z}_t), \quad r \in \mathcal{R}. \end{aligned} \quad (6)$$

The resulting output has resource-by-horizon structure $\hat{\mathbf{Y}}_t \in \mathbb{R}^{R\times|\mathcal{H}|}$. The common encoder is updated by all resource losses; head gradients update only the corresponding head, retaining resource-specific predictive mappings. This single multi-output model introduces no attention, gating, task routing or adaptive sharing.

Let $\mathcal{B}$ be a training mini-batch, and let $\tilde{y}_{i,h}^{(r)}$ and $\tilde{\hat{y}}_{i,h}^{(r)}$ denote the scaled target and prediction for sample i, resource r, and horizon h. M1 uses the equally weighted scaled-space objective

$$\mathcal{L}_{M1} = \frac{1}{|\mathcal{B}|R|\mathcal{H}|} \sum_{i\in\mathcal{B}} \sum_{r\in\mathcal{R}} \sum_{h\in\mathcal{H}} \left(\tilde{\hat{y}}_{i,h}^{(r)} - \tilde{y}_{i,h}^{(r)} \right)^2. \quad (7)$$

This is equivalently the arithmetic mean of the resource-specific mean squared errors, with horizons weighted equally within each resource. It prevents a resource from receiving greater nominal weight merely because its native unit has a larger numerical scale. S1 and S2 use the corresponding single-resource scaled mean squared error. M1 versus S2 can therefore examine representation sharing while both formulations receive the same multichannel input.

The objective fixes the nominal contribution of every resource and horizon; it does not learn task weights or suppress a resource conditionally. Scaling and equal weighting are consequently part of the controlled multi-task definition, whereas the numerical scaling parameters are fitted within the experimental data partitions described later.

3.6. Controlled Information-Sharing Decomposition

The factorial structure defines three comparisons, each evaluated for the same resource, horizon, and matched support. Let $Q(F)$ denote a lower-is-better error functional for formulation F under such a fixed evaluation unit. For a baseline A and candidate B, define the signed error reduction

$$D_Q(A \to B) = Q(A) - Q(B), \quad (8)$$

so that a positive value indicates lower error for B. Exact metric, aggregation, and percentage-normalisation rules are specified in the experimental methodology. The framework-level contrasts are

$$\begin{aligned} C_{\text{context}} &= D_Q(\text{S1} \to \text{S2}), \\ C_{\text{sharing}} &= D_Q(\text{S2} \to \text{M1}), \\ C_{\text{joint}} &= D_Q(\text{S1} \to \text{M1}). \end{aligned} \quad (9)$$

C_{context} compares cross-resource with local-resource input while retaining independently trained target models. C_{sharing} compares shared with separate predictive representations while holding all-resource input available to both formulations. C_{joint} compares the complete joint multi-metric system with the local-information baseline and therefore combines both changes. The first two contrasts prevent the usefulness of additional resource streams from being conflated with the usefulness of a shared learned representation. All three remain empirical matched comparisons: the framework defines what changes between formulations but imposes no expected ordering or sign on their observed errors.

Matching requires the same eligible origins, future targets, resource and horizon coordinates, and evaluation partitions for the formulations entering a contrast. Results are interpreted resource by resource rather than by averaging errors expressed in heterogeneous native units. Because the three raw contrasts are differences of the same error functional Q evaluated on matched support, the S1-to-M1 raw error difference equals the sum of the S1-to-S2 and S2-to-M1 raw differences under the sign convention above. This algebraic identity does not imply that the underlying modelling effects are mechanistically additive or causally isolated. In particular, the baseline-normalised percentage gains used for empirical reporting are not generally additive

because each comparison is normalised by its own reference error. Its role is to characterise the complete change from local, separate forecasting to the shared multi-resource system, while the two intermediate contrasts identify which modelling decision differs.

4. Experimental Methodology

This section instantiates the controlled formulations from Section 3 across one development dataset, two external confirmation environments, and three temporal replication blocks. The design keeps the information-sharing contrasts fixed while adapting only those quantities, such as sample offsets and raw field names, that necessarily depend on the telemetry source.

4.1. Research Questions

The evaluation addresses exactly four research questions. Each question is operationalised by the matched formulation contrasts defined in Section 3.6; it is not answered by comparing unrelated model families or unmatched samples.

RQ1: Does cross-resource context improve forecasting relative to local-resource forecasting? This question is evaluated by comparing S2 with S1, which changes the input from the target resource alone to all four resource histories while retaining separate target-specific models.

RQ2: Does sharing a predictive representation improve forecasting relative to separate contextual models? This question is evaluated by comparing M1 with S2, for which both formulations receive all-resource input but only M1 shares a temporal encoder across target tasks.

RQ3: Does the complete shared multi-metric system outperform local-resource forecasting? This end-to-end question compares M1 with S1 and therefore measures the joint change in input scope and representation sharing.

RQ4: How stable are these effects across resources, environments, temporal blocks, horizons, and evaluation units, and what accuracy–efficiency trade-offs arise? Stability is examined without collapsing the resource-specific contrasts into heterogeneous native-unit errors.

The questions specify estimands rather than expected outcomes. In particular, neither access to additional channels nor parameter sharing is assumed to be beneficial, and efficiency is assessed separately from forecasting accuracy.

4.2. Datasets

Table 2 summarises the experimental environments and their roles. All sources are mapped to the same four resource semantics, but their sampling grids, eligible populations, and evidential roles remain distinct.

4.2.1. TimeTrack Telemetry

TimeTrack is the framework-development source. It contains telemetry from the OpenAirInterface (OAI) CI/CD system. Seven machines were observed between 24 June and 19 July 2024. The canonical panel is the exact sorted intersection of timestamps across the six source files; no nearest-time matching, resampling, interpolation, imputation, or value cleaning is used. Its measured median sampling interval is 42.285164 seconds.

Discontinuities are represented explicitly. A new canonical segment begins when adjacent retained timestamps are separated by more than 84.57 seconds, twice the median interval, yielding nine segments. Historical windows and future targets cannot cross these boundaries. The four study targets are machine-level CPU utilisation, aggregate bidirectional throughput, used memory, and disk-write activity. Throughput is constructed for each machine as the sum of validated `bond0` transmit and receive rates; used-memory and disk-write fields are retained at machine level. TimeTrack supplies internal development evidence and is not treated as an independent external dataset.

4.2.2. Bitbrains fastStorage and Rnd

The Bitbrains fastStorage and Rnd traces are kept as two separate external confirmation environments. CPU, network receive plus transmit throughput, memory usage, and disk-write throughput are mapped respectively to the four common resource targets. Rows that are physically invalid are removed without repair, duplicate timestamps are rejected, and gaps longer than 450 seconds delimit contiguous segments.

Table 2 distinguishes the structural and scale-eligible pools. A prospective training-scale amendment required positive transformed-resource interquartile range in every required training partition. From each restricted pool, VMs were selected by ascending SHA-256 rank of a frozen environment-and-identifier key. The rule is deterministic and uses neither predictive outcomes nor manual substitution. The selected cohorts are therefore samples from scale-estimable, structurally eligible populations; they are not claimed to represent every Bitbrains VM.

4.2.3. Materna Temporal Traces

Materna provides a secondary temporal replication. T1, T2, and T3 are consecutive blocks from the same source, not independent datasets. They are processed independently on their native 300-second grid, with contiguous segments defined per trace and VM. The resource mapping is CPU usage percentage, received plus transmitted network throughput in KB/s, memory usage in KB, and disk-write throughput in KB/s.

Applying the strict training-scale rule jointly across every trace, fold, training phase, context, and resource retains 74 VMs, or 18.5930% of the common structural pool. The first 35 VMs under the frozen ascending SHA-256 rank are used in all three blocks, preserving the same identifiers wherever structurally valid and permitting matched temporal comparisons. T1–T3 are analysed first as separate temporal

Table 2
Evidence sources, study roles and frozen protocol adaptations, distinguishing eligible populations from executed cohorts. Materna T1–T3 are dependent temporal blocks.

Source / block	Study role	Temporal structure / cadence	Eligibility	Executed cohort	Forecast protocol
TimeTrack	Framework development	42.285 s median; 9 gap-bounded segments	Seven machines; 41,356-row canonical panel	7 machines	0.705–90.208 min (eight leads)
Bitbrains fastStorage	External confirmation	Separate environment; 5 min; per-VM segments	916/1,250 structural; 596/916 scale eligible	35 VMs	5, 10, 20, 45, 90 min (five leads)
Bitbrains Rnd	External confirmation	Separate environment; 5 min; per-VM segments	416/500 structural; 272/416 scale eligible	35 VMs	5, 10, 20, 45, 90 min (five leads)
Materna T1–T3	Secondary replication	5 min; blocks analysed separately	398 common structural; 74/398 all-block scale	35 shared VMs	5, 10, 20, 45, 90 min (five leads)

All experiments forecast CPU utilisation, throughput, memory usage and disk-write activity. Materna provides secondary temporal replication: T1–T3 are consecutive dependent blocks from the same source and selected cohort, not independent datasets.

blocks. Any combined estimate uses equal block weights; the blocks are never treated as independent observations in a meta-analysis.

4.3. Target Construction and Preprocessing

Dataset-specific fields and units are mapped to the semantic order CPU, throughput, memory, and disk write. CPU and memory use the identity transform. Throughput is the sum of receive and transmit rates, or the frozen equivalent aggregate in TimeTrack, and uses $\log(1 + x)$. Disk-write activity or throughput uses the same $\log(1 + x)$ transform. These definitions preserve the meaning of each resource while accommodating different source schemas and native units.

At each legal forecasting origin, these mappings yield a four-resource history and future resource-by-horizon target tuple. Direct multi-horizon prediction is used: every head estimates all prescribed future offsets in one forward pass, rather than recursively feeding an earlier prediction into a later horizon. The target construction is identical across formulations on a given dataset. Thus S1 receives only the target channel's history, S2 receives all four histories for the same single target, and M1 receives all four histories and all four target vectors. Missing or invalid values remove the affected origin from common support; they are not repaired differently for one formulation.

For resource r, let g_r denote its fixed transform. Within a legal training partition, transformed values are robustly scaled as

$$\begin{aligned} \tilde{y}^{(r)} &= \frac{g_r(y^{(r)}) - m_r}{q_r}, \\ m_r &= \mathrm{median}_{\mathrm{train}}\, g_r(y^{(r)}), \\ q_r &= \mathrm{IQR}_{\mathrm{train}}\, g_r(y^{(r)}). \end{aligned} \tag{10}$$

The interquartile range must satisfy $q_r > 0$. The final protocol uses no epsilon, fallback scale, or outcome-dependent exception. Inner scalers are fit only on inner-training data; after model selection, final scalers are refit on the complete legal outer-training partition. Validation and test values never contribute to scaler parameters.

CPU and memory predictions are returned to native units by inverse robust scaling. Throughput and disk-write predictions are inverse-scaled, passed through expm1, and deterministically projected onto the non-negative domain. The same transform, scaler, and inverse mapping are used for matched S1, S2, and M1 fits. TimeTrack uses eight output horizons whereas the external datasets use five; this changes output width but not the resource-transform semantics.

4.4. Temporal Support and Leakage Control

Forecasting origins are ordered chronologically within the support universe. An origin is legal only when the maximum candidate context and every required future target are finite and remain in the same contiguous temporal segment. Common support is constructed at the largest context, so shorter contexts gain no additional origins. S1, S2, and M1 consequently use identical origin IDs, target IDs, machines or VMs, folds, horizons, and seeds within every comparison.

Five expanding outer folds are used for each machine, VM, environment, or trace. The first half of the ordered legal origins precedes the first test block; five successive, non-overlapping ten-percent blocks form the outer tests. Earlier test periods may enter the training history of a later fold. For each fold, an otherwise eligible training origin is removed unless its maximum-horizon target timestamp is strictly earlier than the first outer-test origin. This target-time purge prevents a training label from extending into the test period even when its forecasting origin precedes the boundary.

Within the purged outer-training support, the chronologically latest 15% of origins is provisional inner validation. Earlier inner-training origins are purged again against the first validation origin using the maximum forecast horizon. There is no shuffling. Scalers, model parameters, context selection, and epoch selection can use only the relevant legal training and validation partitions; outer-test observations

cannot determine filtering, scaling, context, epoch count, or fitted parameters.

After context and seed-specific epoch selection, each final model is reinitialised and refit once on the complete legal outer-training support with the selected fixed epoch count and a newly fitted outer-training scaler. It has no validation or test callback. Each successful final fit is then evaluated exactly once on its outer-test block. These controls separate model selection from evaluation and ensure that formulation effects are not attributable to different temporal support.

4.5. Forecast Horizons and Contexts

TimeTrack uses sample leads $\{1, 2, 4, 8, 16, 32, 64, 128\}$. At its measured median interval, these correspond approximately to 0.705, 1.410, 2.819, 5.638, 11.276, 22.552, 45.104, and 90.208 minutes. Candidate context lengths are 32, 64, and 128 samples, covering approximately 22.6, 45.1, and 90.2 minutes.

Bitbrains and Materna use five-minute sampling, so their sample leads are $\{1, 2, 4, 9, 18\}$, corresponding exactly to 5, 10, 20, 45, and 90 minutes. Their candidate contexts are 4, 9, and 18 samples, or approximately 20, 45, and 90 minutes. The integer offsets therefore differ by dataset. The adaptation preserves a comparable real-time forecast and context range rather than pretending that one sample index has the same elapsed-time meaning on all grids.

4.6. Model Architecture and Training

All three formulations use the same plain recurrent backbone: a two-layer unidirectional LSTM with hidden size 64, dropout 0.1, and the final recurrent output as its temporal representation. S1 has one input channel and one resource-specific horizon head; S2 has four input channels but remains one independent model per target; M1 has four inputs, one shared encoder, and four independent resource heads. S1 and S2 minimise target-specific scaled mean squared error, whereas M1 minimises the arithmetic mean of the four resource-specific scaled mean squared errors.

Training uses Adam with learning rate 0.001, batch size 256, at most 50 epochs, early-stopping patience 7 during inner fitting, gradient-norm clipping at 1.0, and seeds 0, 1, and 2. No architecture or optimiser search is performed. The plain backbone is deliberately simple because the experiment isolates the information-sharing structure rather than proposing an architecture-specific innovation.

Output width affects parameter counts. For the five-horizon external design, S1 contains 50,757 parameters per resource model (203,028 across four), S2 contains 51,525 per target model (206,100 across four), and M1 contains 52,500 in its single shared model. For eight-horizon TimeTrack, the corresponding counts are 50,952 per S1 model (203,808 across four), 51,720 per S2 model (206,880 across four), and 53,280 for M1. The models are therefore not parameter-equal; accuracy and system footprint are reported as separate properties.

4.7. Context and Model Selection

Context length is the only selected model setting. For S1 and S2, one context is selected independently for each environment or trace, machine or VM, outer fold, and target resource. The objective is the mean inner-validation NMAE over all horizons and the three seeds. M1 selects one context per environment or trace, machine or VM, and fold by taking the equal mean of validation NMAE over the four resources, all horizons, and all seeds. Exact ties are resolved in favour of the shorter context.

Early stopping during each inner fit records a best epoch separately for every seed. Once a context has been selected, the final fit uses that context, the same seed, and its own fixed selected epoch. It starts from a fresh initialisation, refits the resource scalers on all legal outer-training data, and performs no test-based early stopping. Thus context selection does not stand in for broad architecture or hyperparameter tuning, and the final model does not inherit inner-fit weights.

The selection units differ deliberately between independent and shared formulations. S1 and S2 may choose target-specific context lengths because each target has its own model. M1 must choose one common context because a single shared encoder serves all four targets. In both cases, the selection criterion is computed before outer-test evaluation and retains equal horizon and seed contributions.

4.8. Evaluation Metrics and Statistical Analysis

Let y_i, $\hat{y}_i$, and $\bar{y}$ denote the observed value, forecast, and observed mean for n matched cases of one resource and horizon. Evaluation uses

$$\begin{aligned} \mathrm{MAE} &= \frac{1}{n}\sum_{i=1}^{n}|y_i-\hat{y}_i|, \\ \mathrm{RMSE} &= \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}, \\ R^2 &= 1-\frac{\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}{\sum_{i=1}^{n}(y_i-\bar{y})^2}, \\ \mathrm{NMAE}_r &= \frac{\mathrm{MAE}_r}{q_r}, \end{aligned} \tag{11}$$

where q_r is the native-unit training IQR for resource r. MAE and RMSE are reported in each resource's native units; R^2 and NMAE are unitless. Native errors are never averaged across different resources.

Persistence supplies a separate absolute forecasting sanity check:

$$\hat{y}^{(r),\mathrm{pers}}_{t+h} = y^{(r)}_{t}, \qquad \mathrm{Skill}_{r,h} = 1-\frac{\mathrm{MAE}^{\mathrm{model}}_{r,h}}{\mathrm{MAE}^{\mathrm{pers}}_{r,h}}. \tag{12}$$

Persistence skill does not replace the controlled formulation comparisons: relative improvement over another learned formulation does not imply absolute superiority over persistence.

The primary aggregation begins by averaging stochastic-model MAE across the three seeds within each matched

environment or trace, VM, fold, resource, horizon, and formulation cell. For a fixed resource and horizon, those seed-averaged VM–fold cells receive equal weight to form MacroMAE. A resource-overall MacroMAE then gives equal weight to every horizon. For candidate C and baseline B, the reported gain is

$$G(C : B) = 100 \frac{\text{MacroMAE}(B) - \text{MacroMAE}(C)}{\text{MacroMAE}(B)}. \quad (13)$$

A positive value indicates lower candidate error. The three contrasts are S2 versus S1, M1 versus S2, and M1 versus S1. Secondary diagnostics include pooled prediction-level gain, mean and median matched-cell relative gain, and the fractions or counts of positive VMs, folds, horizons, and seed aggregates.

Prospectively frozen directional classifications combine overall gain magnitude with horizon, VM, fold, and seed consistency. Their exact thresholds and the high-level RQ1–RQ4 decision logic are preserved in the supplementary decision-criteria source; thresholds are not adapted after observing outcomes. For external uncertainty, a paired VM-cluster bootstrap samples 35 VMs with replacement, retains all five folds and the paired formulation structure, and operates on seed-averaged matched MAEs. It uses 10,000 replicates, percentile 95% intervals (2.5th and 97.5th percentiles), and random seed 20261007. The bootstrap is performed independently within each Bitbrains environment and each Materna trace. Materna blocks are not resampled as independent datasets, and this exact external bootstrap is not retrospectively imposed on earlier exploratory TimeTrack analyses.

4.9. External and Temporal Replication Design

The study assigns distinct prospective roles to each source. TimeTrack is the framework and development study. Bitbrains fastStorage and Rnd are separate external confirmation environments; each is analysed before any equal-environment synthesis. Materna is a secondary temporal replication in which T1, T2, and T3 probe stability across consecutive blocks from one source. This progression holds the S1/S2/M1 factorial, semantic resource mapping, controlled contrasts, matched-support principle, and approximately comparable real-time forecast range constant while allowing source-specific field names, units, and sample offsets.

No numeric score pools all dataset roles. Estimates for each Bitbrains environment precede an unweighted equal-environment summary where applicable. Materna likewise reports each trace first. Its descriptive synthesis for any fixed comparison and resource is

$$G_{\text{temporal}} = \frac{G_{\text{T1}} + G_{\text{T2}} + G_{\text{T3}}}{3}. \quad (14)$$

This equal-temporal-block estimate is not weighted by rows, origins, predictions, duration, or fit count. It is explicitly not an independent-dataset meta-analysis. Cross-source synthesis is therefore descriptive and based on direction and frozen classification consistency, rather than a pooled error or nominal interval that would erase the sources' different roles and dependence structures.

4.10. Population Restrictions

External eligibility requires both structural validity and non-degenerate training variability. Specifically, every transformed CPU, throughput, memory, and disk-write series must have IQR strictly greater than zero in every required scaler-fit partition. The partitions cover each fold, inner and final training phase, and candidate context; for Materna the same requirement also holds in every temporal block. There is no epsilon, fallback, resource mask, or formulation-specific exemption.

Table 2 reports the structural, scale-eligible and executed populations. Deterministic hash ranking follows eligibility filtering; selection is neither random nor performance-ranked. Accordingly, external conclusions concern eligible dynamic, scale-estimable, multi-resource VM populations. They do not automatically generalise to excluded idle, quiescent, resource-constant, intermittently observed, or otherwise structurally invalid VMs, nor to every VM in the source archives. This qualification is part of the estimand itself: deterministic selection makes the executed cohort reproducible, but it does not convert the cohort into a probability sample of the rejected or unobserved populations.

5. Results

Results are reported as the primary overall MacroMAE gain defined in Section 4.8. Positive values mean that the candidate formulation has lower MacroMAE than its baseline; negative values therefore indicate degradation. Figure 2 gives the complete resource-level primary result for every development, external, and temporal evidence block. The prospective VM-cluster intervals are used selectively below where they distinguish a directionally clear effect from an uncertain near-zero result. TimeTrack did not use the same prospective external bootstrap and is therefore reported without those intervals.

The three evidence roles are kept distinct throughout this section. TimeTrack provides the development result, the two Bitbrains environments provide the external-environment results, and the three Materna traces provide the prospective temporal results. Each reported gain retains the aggregation and baseline fixed for its own evidence block. Comparisons between blocks are therefore descriptive: no prediction-level pooling is used, and the six blocks are not treated as statistically independent datasets. The frozen qualitative classes summarise consistency across the prescribed lower-level units, while the VM-cluster intervals quantify uncertainty for the external and temporal blocks. Neither replaces the primary resource-overall MacroMAE gain.

5.1. RQ1 Cross-Resource Context

RQ1 compares S2 with S1 and changes only the histories available to each target-specific model. Figure 2(a) shows selective contextual value, not uniform improvement. In

(a) Contextual input: S2 versus S1

	TimeTrack	fastStorage	Rnd	Materna temporal blocks T1	T2	T3
CPU	-3.31	-3.20	+1.41	-4.32	+4.21	-2.79
Throughput	-0.24	+0.01	-7.21	+0.99	+0.29	+0.04
Memory	+0.86	+2.45	+1.67	-0.02	+3.95	+3.09
Disk write	-0.49	+2.69	+0.16	-6.70	-1.12	+1.91

(b) Shared representation: M1 versus S2

	TimeTrack	fastStorage	Rnd	Materna temporal blocks T1	T2	T3
CPU	+0.47	+0.05	-16.51	+0.88	-2.56	-8.10
Throughput	-0.84	-27.09	-14.32	-7.31	-8.69	-7.91
Memory	-18.48	-4.87	-8.92	-13.22	-9.27	-8.70
Disk write	-0.80	-4.07	-5.55	-27.46	-7.89	-71.30

(c) Full joint system: M1 versus S1

	TimeTrack	fastStorage	Rnd	Materna temporal blocks T1	T2	T3
CPU	-2.83	-3.15	-14.86	-3.40	+1.75	-11.11
Throughput	-1.08	-27.08	-22.57	-6.25	-8.37	-7.87
Memory	-17.46	-2.30	-7.09	-13.24	-4.95	-5.34
Disk write	-1.29	-1.27	-5.38	-36.00	-9.10	-68.03

MacroMAE gain (%) + Lower candidate error - Higher candidate error Sign-only fill

Figure 2: Primary overall MacroMAE percentage gains for (a) S2 versus S1, (b) M1 versus S2 and (c) M1 versus S1 across the evaluated evidence blocks. Positive values indicate lower MacroMAE for the candidate formulation; negative values indicate higher MacroMAE. Cell fills encode sign only, not magnitude. Values are point estimates, not significance classifications. Materna T1, T2 and T3 are dependent temporal blocks. Comparisons are bounded to the evaluated implementation and eligible populations.

TimeTrack, memory was the sole positive resource-level effect, while all four frozen classifications remained neutral or mixed because the required consistency thresholds were not met.

The Bitbrains environments did not reproduce one common direction. In fastStorage, memory was broadly positive and disk write conditionally positive; CPU and throughput were neutral or mixed. The memory bootstrap interval excluded zero, whereas the near-zero throughput interval crossed it. In Rnd, CPU and memory were conditionally positive, while throughput and disk write were neutral or mixed. All four Rnd intervals crossed zero, including the interval for its negative throughput point estimate.

The Materna blocks further changed the resource pattern. T1 memory was approximately neutral and slightly negative (-0.02%); only throughput reached the conditionally positive class. The T1 CPU interval was entirely negative, while the other three intervals crossed zero. T2 CPU and memory were broadly positive with intervals above zero; throughput and disk write remained neutral or mixed. T3 memory and disk write were conditionally positive, but all four T3 intervals crossed zero.

Memory therefore has the clearest repeated positive contextual pattern, but not universally. CPU, throughput and disk write change direction across blocks. The resource ordering also changes: the strongest contextual benefit does not remain attached to one resource, while Rnd throughput and T1 disk write illustrate adverse effects on resources that are near zero or positive elsewhere. Contextual value varies by resource, environment and time, not merely in magnitude around a stable ordering.

5.2. RQ2 Shared-Representation Transfer

RQ2 compares M1 with S2, holding the all-resource input scope fixed and changing whether the predictive representation is shared. Throughput, memory and disk write had negative primary gains in all six blocks (Figure 2(b)). CPU was positive only in TimeTrack, fastStorage and Materna T1. These small or near-zero exceptions do not support uniformly beneficial sharing within the evaluated M1 design.

In TimeTrack, memory was broadly negative; the remaining frozen resource classifications were neutral or mixed. In fastStorage, CPU was neutral or mixed and its bootstrap interval crossed zero. Throughput, memory and disk write were broadly negative with intervals below zero. Rnd was broadly negative for every resource, and each interval also lay below zero.

Materna T1 CPU was neutral or mixed with an interval crossing zero, whereas the other resources were broadly negative with intervals wholly below zero. T2 was broadly negative for all four resources, with entirely negative overall intervals. In T3, throughput, memory and disk write were broadly negative; CPU was neutral or mixed under the frozen consistency rule despite its negative primary gain. All four T3 intervals excluded zero, with the disk-write interval spanning $[-239.63\%, -5.94\%]$.

Within the controlled shared-LSTM formulation, degradation therefore recurred outside the CPU exceptions, but its magnitude remained resource- and block-dependent. External and temporal effects for the other resources often combined sizeable degradation with a broadly negative class and an interval below zero. A negative point estimate and a consistently negative result remain distinct under the frozen protocol.

5.3. RQ3 Full Joint-System Performance

RQ3 compares the complete M1 system with local-resource S1 models. Figure 2(c) is negative in every displayed cell except Materna T2 CPU, so the full joint formulation generally did not outperform S1. In TimeTrack, memory and disk write were broadly negative; CPU and throughput were neutral or mixed under the prospective consistency criteria.

In fastStorage, CPU and throughput were broadly negative, whereas memory and disk write were neutral or mixed. The throughput interval was entirely below zero, while the other three overall intervals crossed zero. In Rnd, every resource was broadly negative and every interval lay below zero.

In Materna T1, throughput and disk write were broadly negative; CPU and memory were neutral or mixed, although all four VM-cluster intervals lay below zero. The T2 CPU exception (+1.75%) was conditionally positive, but its interval crossed zero ($[-3.52\%, +6.07\%]$). T2 disk write was broadly negative, and throughput and memory were neutral or mixed. In T3, CPU, throughput and disk write were broadly negative, while memory was neutral or mixed; only the memory interval narrowly crossed zero. These differences in classification and uncertainty remain important despite the nearly uniform negative sign pattern.

The complete-system comparison incorporates both the move from local to all-resource inputs and the move from separate to shared representation. Its observed gain is therefore reported directly rather than reconstructed by adding the RQ1 and RQ2 percentages. The block-level evidence is not pooled or treated as six statistically independent datasets.

5.4. RQ4 Resource, Environment, and Temporal Heterogeneity

Figure 2 reveals heterogeneity at each prespecified level. Resource heterogeneity is evident in the selective S2-versus-S1 benefits and changing resource ordering. CPU, throughput and disk write change direction across evidence blocks, whereas memory is positive in five and nearly zero and negative only in Materna T1. Repeated positive memory effects therefore do not imply invariant resource behaviour.

External-environment heterogeneity is stronger for representation sharing: fastStorage CPU was near zero while the other three resources were negative; Rnd was broadly negative for all four. Temporal heterogeneity within the dependent Materna blocks is distinct: M1-versus-S2 CPU changed from positive in T1 to negative in T2–T3, moving from neutral or mixed to broadly negative and back to neutral or mixed. Throughput, memory and disk write were broadly negative in all three Materna blocks, with varying magnitudes. For M1 versus S1, CPU alone reversed sign in T2; throughput and disk write stayed negative and memory stayed negative but neutral or mixed across T1–T3.

Sign, magnitude, and frozen class capture different aspects of this heterogeneity. For example, T3 S2-versus-S1 throughput was slightly positive at +0.04% but remained neutral or mixed, while T2 CPU was both positive and broadly positive. T3 M1-versus-S2 CPU had a negative point estimate and an interval below zero, yet remained neutral or mixed under the separate consistency rule. These combinations prevent near-zero sign changes from being treated as equivalent to changes supported across the prescribed units. They also show why the within-Materna decision uses the frozen classification and trace rules alongside the raw gains.

The frozen within-Materna RQ4 outcome was recorded as follows.
`R4_TEMPORAL_HETEROGENEITY_CONFIRMED`. Mechanically, seven of the 12 resource–comparison pairs changed frozen classification across T1–T3 and five changed raw sign. The S2-versus-S1 trace decision also changed: T1 met the frozen non-support rule, whereas T2 and T3 were mixed. The M1-versus-S2 trace-level decision was consistently not supported, even though its CPU classification varied. These prespecified changes satisfy the RQ4 criterion; no post-hoc temporal regimes or causal mechanisms are inferred.

5.5. Persistence Boundary

Relative formulation gains do not establish superiority to a persistence forecast. In TimeTrack, S1, S2, and M1 all had positive mean persistence skill for throughput and disk write, and their CPU means were also positive. Memory was the boundary: mean skill was −0.1272 for S1, −0.1494 for S2, and −0.6047 for M1, with positive skill at only two, four, and zero of eight leads. Thus the large negative M1-versus-S2 memory result coincided with weak absolute performance relative to persistence, whereas the throughput and disk-write formulation comparisons occurred in resources with positive average absolute skill.

Neither Bitbrains environment contained a formulation that beat persistence for every resource. Overall CPU skill was negative for S1, S2, and M1 in both environments. In fastStorage, throughput skill was also negative for every formulation; memory and disk write were positive for all three. Rnd again had positive memory and disk-write skill for all formulations, while throughput was only slightly positive for S1 (0.0384) and negative for S2 (−0.0257) and M1 (−0.1176). Consequently, a positive S2-versus-S1 gain for Rnd CPU or a relative degradation under M1 does not by itself describe absolute forecasting skill.

Materna showed a different aggregate boundary. S2 beat persistence in all 12 trace–resource overall aggregates. No formulation, however, beat persistence in every individual trace–resource–lead cell. At the overall level, M1 failed the check for T1 throughput (−0.0608), T1 disk write (−1.7590), T3 throughput (−0.2629), and T3 disk write (−1.3493); it

Table 3
Model consolidation and observed Materna fitting cost.

Formulation	Models	Parameters	Fit-hours
S1	4	203,028	85.59
S2	4	206,100	75.30
M1	1	52,500	20.41

Parameter counts refer to the five-horizon external instantiation. Fit-hours are observed totals from the three Materna traces. Relative to S2, M1 uses approximately 74.5% fewer parameters and 75% fewer models.

remained positive for all four T2 resources. S1 additionally failed only T1 throughput at the overall level, while S2 had no overall failures. These persistence results delimit absolute forecasting performance and remain separate from the controlled S1–S2–M1 comparisons.

Three boundaries follow directly from these observations. First, a positive gain answers only whether one learned formulation improves on another; it does not answer whether either improves on persistence. Second, an overall positive skill does not require every horizon-level cell to be positive, as the Materna lead-level result demonstrates. Third, absolute skill remains resource dependent: memory and disk write were the stronger Bitbrains cases, whereas CPU remained below persistence despite some positive relative formulation gains. Reporting the persistence check beside the formulation comparison therefore prevents a relative improvement in an absolutely weak setting from being described as general forecasting superiority.

5.6. Accuracy-Efficiency Trade-Off

The shared M1 construction substantially consolidated the evaluated external system. In the five-horizon instantiation, S1 comprised four models and 203,028 parameters, S2 four models and 206,100 parameters, and M1 one model and 52,500 parameters. Relative to S2, M1 therefore used approximately 74.5% fewer parameters and 75% fewer models. These counts are not universal architecture constants: the eight-horizon TimeTrack instantiation contained 206,880 S2 parameters and 53,280 M1 parameters.

Observed Materna fitting time gave the same operational ordering. Across the three traces, S1 required 85.594240 fit-hours, S2 75.303186 fit-hours, and M1 20.405083 fit-hours. These measurements are execution-cost evidence for the Materna workload, not a universal runtime benchmark. The consolidation is therefore substantial, but the RQ2 and RQ3 results show that, for the controlled M1 design, it was accompanied by predictive degradation across most evaluated resource–environment and resource–trace comparisons. Efficiency and forecasting accuracy remain distinct properties of the formulations.

The two efficiency measures capture different consolidation mechanisms. The model-count change removes three separately trained target-specific models, whereas the parameter change compares the combined parameter inventories of the five-horizon external implementations. Materna fit-hours add observed execution accounting for the full frozen run, including the formulation-level training workload recorded by that protocol. All three measures favour M1 on resource use, but none modifies the accuracy estimand or offsets a negative MacroMAE gain. The trade-off is therefore stated as co-occurring consolidation and predictive degradation for this implementation, without extrapolating its timings to other hardware, horizon sets, or shared architectures.

6. Cross-Dataset Synthesis

The source summaries in Table 4 preserve the distinct evidential roles defined in Section 4.9. They use the frozen equal-environment Bitbrains and equal-temporal-block Materna summaries, without averaging either with TimeTrack. Comparisons are descriptive, considering direction, magnitude and frozen classifications (Section 5), not a global cross-dataset score, cross-source confidence interval or pooled statistic. Agreement indicates replication at the displayed source-summary level; disagreement marks a conditional boundary. Neither changes the within-source estimand or prespecified block weights. The conclusions introduce no ranking, cross-source uncertainty calculation or new global effect estimate.

6.1. TimeTrack

TimeTrack established the initial boundary between access to additional resource histories and sharing a predictive representation. For S2 versus S1, the contextual effect was mixed and mostly weak: memory was the sole positive resource-overall gain at +0.86%, while CPU, throughput, and disk write were negative. This development result did not support a broad contextual advantage and instead motivated resource-specific external evaluation.

Holding the all-resource input scope fixed, M1 did not improve broadly on S2. The small positive CPU gain was an exception, while throughput and disk write were negative and memory degraded by −18.48%. The complete M1-versus-S1 comparison was negative for all four resources. TimeTrack therefore supplied the initial evidence that having cross-resource information and learning one shared representation are empirically distinct design choices. Its role is framework development rather than independent confirmation; the external and temporal results below test whether those directional boundaries persist.

6.2. Bitbrains External Confirmation

The frozen Bitbrains synthesis assigns equal weight to fastStorage and Rnd. The external evidence did not establish a universal contextual benefit: memory and disk write improved on average for S2 versus S1, whereas CPU and throughput did not. The separate environment results in Section 5.1 further show that the direction was not invariant between fastStorage and Rnd.

Representation sharing produced a more consistent external pattern: M1-versus-S2 and M1-versus-S1 gains were negative for all four resources. These summaries confirm

Table 4
Frozen descriptive source summaries of primary overall MacroMAE gain (%).

	S2 versus S1			M1 versus S2			M1 versus S1		
Resource	TimeTrack	Bitbrains	Materna	TimeTrack	Bitbrains	Materna	TimeTrack	Bitbrains	Materna
CPU	−3.31	−0.89	−0.97	0.47	−8.23	−3.26	−2.83	−9.01	−4.25
Throughput	−0.24	−3.60	0.44	−0.84	−20.71	−7.97	−1.08	−24.82	−7.50
Memory	0.86	2.06	2.34	−18.48	−6.89	−10.40	−17.46	−4.70	−7.85
Disk write	−0.49	1.42	−1.97	−0.80	−4.81	−35.55	−1.29	−3.33	−37.71

Positive values indicate lower MacroMAE for the candidate formulation; negative values indicate higher MacroMAE. TimeTrack: development summary. Bitbrains: equal-environment synthesis of fastStorage and Rnd. Materna: equal-temporal-block synthesis of T1–T3, consecutive dependent temporal blocks rather than independent replications. Comparisons are descriptive, without cross-source aggregation.

that the evaluated shared-LSTM representation did not convert all-resource input access into broad external gains. They also retain the distinction between a mixed contextual-input result and a consistently unfavourable sharing result, rather than reducing both comparisons to one multivariate-model conclusion.

6.3. Materna Temporal Replication

The frozen Materna synthesis gives T1, T2, and T3 equal weight while preserving their status as consecutive dependent blocks. For S2 versus S1, memory again provided the clearest contextual benefit, throughput was only weakly positive, and CPU and disk write were negative. Together with the sign and classification changes documented in Section 5.4, these values confirm that the contextual effect depends on both resource and temporal block.

M1-versus-S2 and M1-versus-S1 gains were negative for all four resources. Materna therefore replicated the negative shared-representation and complete-system results while also confirming temporal heterogeneity in the contextual effect. This conclusion concerns the prespecified equal-block synthesis; it neither treats the three traces as independent datasets nor averages their prediction rows.

6.4. Generalisation Boundaries

Cross-resource context. Across the evaluated sources, cross-resource context was selective. It was not universally beneficial. S2 versus S1 changed direction across resources, Bitbrains environments, and Materna blocks. Memory supplied the strongest repeated positive contextual result: its primary gain was positive in TimeTrack, fastStorage, Rnd, Materna T2, and Materna T3. Materna T1 was the exception at −0.02% before rounding, an effectively neutral negative value under the frozen classification. This pattern strengthens a resource-, environment-, and time-dependent benefit boundary. It does not establish memory as universally improved, nor does it imply that cross-resource inputs are generally harmful.

Shared representation. The evaluated M1 representation did not reliably turn access to all resource histories into predictive gains. Relative to S2, throughput, memory, and disk write were negative in TimeTrack, both Bitbrains environments, and all three Materna blocks. CPU contained limited positive exceptions, but its direction and classification were not stable. This repeated contrast strengthens the boundary between supplying cross-resource information to separate target models and sharing the learned representation itself. The conclusion is specific to the controlled M1 shared-LSTM design; it does not assert that all multi-task or shared-representation architectures must behave similarly.

Complete joint system. M1 did not broadly outperform the local S1 baseline. Its disk-write gains relative to S1 were negative in TimeTrack, fastStorage, Rnd, T1, T2, and T3. Their frozen descriptive six-block arithmetic mean is −20.18% (−20.178231% before rounding). This quantity is not a pooled effect, confidence-weighted estimate, or mean of independent datasets; it is reported only as the prespecified descriptive summary of six displayed blocks. More broadly, every resource–block M1-versus-S1 gain in Figure 2 was negative except Materna T2 CPU. The repeated pattern therefore strengthens the conclusion that the complete joint system underperformed its local-resource baseline within the evaluated design.

The accuracy boundary coexists with a practically meaningful efficiency gain. As reported in Section 5.6, M1 uses approximately 74.5% fewer parameters and 75% fewer models than S2 in the five-horizon external instantiation. Compactness therefore improved substantially, but it did not translate into broadly improved forecasting across the evaluated sources and temporal blocks.

Taken together, the development, external, and temporal evidence strengthens the three frozen information-sharing boundaries and supports the study as a multi-dataset temporal boundary analysis. Its population scope remains the eligible dynamic multi-resource VMs defined in Section 4.10. In particular, the Bitbrains and Materna cohorts exclude VMs lacking nondegenerate four-resource training variability across required scaler-fit partitions, and the Materna blocks remain dependent observations from one source. The synthesis therefore does not claim unrestricted generalisation to all cloud systems, multivariate forecasters, or shared architectures.

7. Discussion

7.1. When Cross-Resource Context Helps

Section 5.1 motivates validating contextual inputs per target, environment and time rather than assuming that additional telemetry helps. The practical question is whether other histories improve the chosen target in its operating context, not which resource should always receive them.

MMRP's univariate/multivariate single-target results likewise do not uniformly favour multivariate inputs (Tripathi et al., 2025). This is a meaningful precedent for the contextual comparison, although its reported feature and selection details do not establish equivalence to our matched protocol. Resource-forecasting and transfer experiments also make the evaluation environment an explicit design consideration (Rossi et al., 2025).

TimeTrack's contemporaneous associations (Meliani et al., 2025) motivate candidate inputs but do not establish incremental predictive information conditional on the target's own history. The comparisons neither establish causal resource coupling nor explain observed directions through intrinsic memory behaviour, throughput noise or disk-write sparsity; these explanations require additional measurements.

7.2. Shared Representation and Negative Transfer

Without independent all-input S2 models, the complete M1-versus-S1 comparison changes input scope and representation sharing together. S2 is therefore an important intermediate comparator, making contextual-input outcomes separately observable from the implemented sharing transition under the matched operational contrasts (Section 5.2).

Multi-task learning permits useful inductive transfer (Caruana, 1997), but task-grouping research evaluates which combinations benefit (Standley et al., 2020; Fifty et al., 2021). Forecasting studies already examine negative transfer and selective sharing (Nikentari and Wei, 2022; Nochumsohn et al., 2025); our contribution is a scoped runtime-resource comparison, not discovery of the phenomenon.

The observed M1 degradation is operationally consistent with adverse transfer, but the experiment does not identify the mechanism. Aggregate capacity, model count, joint optimisation and common versus target-specific context selection also differ between M1 and S2. These are matched operational contrasts, not a parameter-matched causal test of sharing alone. Gradient conflict, representation interference, capacity competition and task covariance were not measured. Multi-objective and gradient-aware approaches offer alternative training designs (Sener and Koltun, 2018; Yu et al., 2020); their existence does not diagnose our models. M1 is deliberately a conventional shared-LSTM formulation. Its repeated adverse outcomes justify testing a common representation before adopting it, not rejecting multi-task forecasting or asserting that separate models are always preferable.

7.3. Implications for Runtime and Cloud Software Systems

Monitoring and forecasting inform software adaptation and resource management (Taherizadeh et al., 2018; Bauer et al., 2020). Here, telemetry selection and cross-target modelling should be separate decisions: additional channels require validation, and a target benefiting from context may still favour independently fitted prediction.

Evaluate local, contextual-independent and shared formulations separately rather than treating "multivariate" as one category. Assess intended resources and horizons under future-only validation, retaining environment- and time-specific results. Relative gains do not establish absolute skill (Section 5.5). This process neither prescribes one architecture for every resource nor replaces system-specific validation.

Forecast-design evidence remains distinct from operational effectiveness. Autoscaling research places prediction within a wider control loop (Qu et al., 2018). We deployed no autoscaler, scheduler or controller and measured no SLA improvement, monetary saving, energy reduction, capacity-planning benefit or incident prevention. Those outcomes require separate evaluation of forecasts together with decision policies and their operating constraints.

7.4. Accuracy-Efficiency Trade-Off

Table 3 shows substantial consolidation: in the five-horizon external instantiation, M1 uses approximately 74.5% fewer parameters and 75% fewer models than S2, alongside lower observed Materna fitting workload. Section 5.6 nevertheless records co-occurring accuracy losses. Consolidation should therefore be evaluated as a trade-off, not a free accuracy gain. Joint consideration of predictive quality and computational requirements is established in task grouping and software-service forecasting (Standley et al., 2020; Syu et al., 2017).

Counts and fit-hours characterise this implementation and workload, not inference latency, deployed RAM consumption, energy or monetary cost. Parameter reductions need not yield proportionate savings. Choosing a compact model requires resource-specific error tolerances and direct measurements on the intended platform; model count alone cannot establish a production saving.

7.5. Applicability and Generalisation Limits

These implications concern four resource semantics and a conventional two-layer LSTM, within particular runtime/cloud contexts. The external populations are eligible dynamic multi-resource VMs, not unrestricted archive samples. Bitbrains and Materna require nondegenerate training IQR across all required scale-fit partitions; only 74 of Materna's 398 structurally common VMs pass strict all-trace scale eligibility. The executed cohorts contain 35 selected VMs per Bitbrains environment and 35 shared Materna identifiers. Section 4.10 explains why these findings do not automatically extend to excluded idle or resource-constant VMs. Materna T1–T3 remain dependent temporal blocks, not three independent datasets.

The results motivate testing selective resource grouping, adaptive sharing and hybrid shared/task-specific representations. Task-grouping and gradient-aware methods provide concrete candidates (Fifty et al., 2021; Yu et al., 2020; Nochumsohn et al., 2025). Global forecasting research and runtime foundation-model comparisons also motivate testing these alternatives (Montero-Manso and Hyndman, 2021; Di Menna et al., 2026). None of these alternatives, including mixture-of-experts methods, was tested here, and none can be assumed to repair M1's degradation. Future comparisons should retain independent all-input controls and resource-specific assessment while measuring the mechanisms they seek to explain.

8. Threats to Validity

8.1. Construct Validity

The three arms are not a complete 2×2 factorial. S1/S2 differ in input dimensionality and parameter count; S2/M1 differ in model count, aggregate capacity, optimisation and target-specific versus common context selection. The latter is therefore an operational shared-representation contrast, not a parameter-matched causal intervention (Section 3.6). Its interpretation includes these implementation differences, rather than attributing every change to shared parameters alone.

CPU utilisation, throughput, memory usage and disk-write activity are semantic mappings across datasets. Source definitions and native units differ, so the mapping does not establish perfect measurement equivalence (Section 4.3). Primary comparisons use MacroMAE-based gains; RMSE, R^2, NMAE and persistence supply complementary diagnostics. Metric choice may change apparent rankings (Hyndman and Koehler, 2006). Accordingly, the reported accuracy boundary concerns the specified measures, not every possible forecasting objective.

8.2. Internal Validity

Temporal evaluation design affects forecasting performance estimation (Cerqueira, Torgo and Mozetič, 2020; Hyndman and Athanasopoulos, 2021). Section 4.4 specifies chronological origins, gap-bounded legal segments, expanding outer folds and maximum-horizon purging. Inner validation remains inside outer-training support, scalers use training data only, and outer tests do not select contexts or epochs. Matched support across S1/S2/M1 reduces differences attributable to evaluation samples. External and temporal criteria were fixed prospectively. These controls cannot exclude every implementation or data-provenance error.

TimeTrack's large gap is handled through segmentation: windows and targets do not cross it, but available contiguous support is reduced. Three seeds limit dependence on one initialisation, while residual neural optimisation stochasticity remains. Selected contexts and epochs may interact with formulation. The models are not capacity matched, hyperparameter search is intentionally limited, and the plain LSTM may favour or disfavour particular sharing patterns (Sections 4.6 and 4.7).

8.3. External Validity

TimeTrack, Bitbrains fastStorage and Rnd, and Materna T1–T3 represent particular runtime/cloud contexts, not all workloads. Scale eligibility retains 596 of 916 structurally eligible fastStorage VMs, 272 of 416 Rnd VMs, and 74 of 398 structurally common Materna VMs. The strict rule requires nondegenerate training IQR across every required resource/scaler partition, including every Materna block. Only 35 VMs are selected per external environment or cohort (Section 4.10). Deterministic hash selection is reproducible but is not probability sampling. Conclusions therefore concern eligible dynamic multi-resource VM populations, not every archived VM, particularly excluded idle or resource-constant cases.

Materna's consecutive blocks share the source and selected VM population, providing repeated-within-source evidence rather than independent datasets (Section 4.9).

Only the controlled two-layer LSTM formulations are evaluated, including the conventional shared M1 design. This does not establish behaviour for Transformers, mixture-of-experts, selective task grouping, gradient surgery, adaptive sharing or foundation forecasters. Only four resource semantics are evaluated. Findings do not extend directly to latency, energy, queue lengths, request rates, application QoS or SLA outcomes.

8.4. Conclusion Validity

MAEs are averaged over seeds within matched cells before equal weighting of VM–fold cells and horizons (Section 4.8). External uncertainty uses paired VM-cluster bootstrap intervals. These intervals condition on the evaluated cohorts and protocol; they do not establish universal population inference. Conclusions consider magnitude and frozen consistency criteria rather than one aggregate number. Nevertheless, multiple resource/horizon comparisons create many opportunities for variation. Intervals and classifications are not presented as classical null-hypothesis significance claims.

Persistence distinguishes gains over another learned formulation from absolute forecasting skill (Section 5.5). Bitbrains and Materna syntheses are descriptive equal-environment and equal-temporal-block summaries, respectively. The six-block disk-write mean in Section 6 is likewise descriptive only, not an effect estimated from independent datasets. No cross-dataset inferential meta-analysis is performed. This is formulation- and population-bounded evidence, not a universal statistical verdict on resource sharing.

9. Conclusion

Multi-metric runtime forecasting involves two distinct decisions: whether a predictor receives other resources' histories, and whether resource targets share a learned predictive representation. S2 versus S1 evaluates contextual information, M1 versus S2 implemented representation sharing,

and M1 versus S1 the complete joint-system change. These operational contrasts were examined through TimeTrack development, Bitbrains external environments and Materna dependent temporal replication.

Cross-resource context was selective, varying with resource, environment and time. Memory showed the clearest repeated contextual benefit, but not in every block. For the evaluated M1 shared-LSTM formulation, throughput, memory and disk write had adverse M1-versus-S2 effects across all six evidence blocks; CPU contained limited exceptions. The complete M1 system generally did not outperform local S1 forecasting, with Materna T2 CPU the exception. These accuracy losses coexisted with substantial consolidation: in the five-horizon external instantiation, M1 used approximately 74.5% fewer parameters and 75% fewer models than S2. The result is an accuracy–efficiency boundary for the evaluated implementation, not a general rejection of shared modelling.

Cross-resource histories should be validated per target, and shared representations assessed separately against independently trained contextual models. The scope is the four evaluated resource semantics and conventional shared-LSTM formulation, with external conclusions restricted to eligible dynamic VM populations. The evidence does not establish performance for all cloud systems, alternative multi-task architectures or production controllers. Future work should test whether selective or adaptive sharing can retain consolidation benefits while avoiding the adverse transfer observed here.

Data availability

TimeTrack is publicly available on Kaggle (Apache 2.0 in its dataset metadata) and in the publisher's Zenodo archive (doi:10.5281/zenodo.15442251; CC BY 4.0); its description paper is Meliani et al. (2025). These are source-specific licence statements. Bitbrains fastStorage and Rnd are publicly available as GWA-T-12 (Shen, van Beek and Iosup, 2015); Materna T1–T3 are publicly available as GWA-T-13 (Kohne, Spohr, Nagel and Spinczyk, 2014; Kohne, Pasternak, Nagel and Spinczyk, 2016). We acknowledge Bitbrains IT Services Inc. and Materna GmbH Information & Communications, respectively, for providing these traces through the Grid Workloads Archive. Their separate source pages specify acknowledgement and citation conditions, rather than a standard open-data licence. Readers should obtain third-party raw data from these original sources and consult each source's terms before redistribution.

Source code, protocols, derived analysis outputs and SHA-256 inventories are publicly accessible in the project repository at scientific checkpoint `fee20c3`. The public checkpoint does not include the full fitted-model or per-fit prediction collections; the checksum inventories do not themselves provide those files. No repository-wide reuse licence is asserted here.

CRediT authorship contribution statement

Abdelhak Kadouma: Conceptualization, Methodology, Software, Data curation, Formal analysis, Validation, Visualization, Writing – original draft. **Amit Shukla:** Supervision, Conceptualization, Methodology, Writing – review & editing. **Mohammed Elmusrati:** Supervision, Project administration, Writing – review & editing.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

Bauer, A., Züfle, M., Herbst, N., Zehe, A., Hotho, A., Kounev, S., 2020. Time series forecasting for self-aware systems. Proceedings of the IEEE 108, 1068–1093. doi:10.1109/JPROC.2020.2983857.

Bi, J., Yuan, H., Zhou, M., Liu, Q., 2019. Time-dependent cloud workload forecasting via multi-task learning. IEEE Robotics and Automation Letters 4, 2401–2406. doi:10.1109/LRA.2019.2899224.

Calheiros, R.N., Masoumi, E., Ranjan, R., Buyya, R., 2015. Workload prediction using ARIMA model and its impact on cloud applications' QoS. IEEE Transactions on Cloud Computing 3, 449–458. doi:10.1109/TCC.2014.2350475.

Caruana, R., 1997. Multitask learning. Machine Learning 28, 41–75. doi:10.1023/A:1007379606734.

Cerqueira, V., Torgo, L., Mozetič, I., 2020. Evaluating time series forecasting models: an empirical study on performance estimation methods. Machine Learning 109, 1997–2028. doi:10.1007/s10994-020-05910-7.

Di Menna, F., Traini, L., Cortellessa, V., 2026. Forecasting software runtime metrics: A comparative study of classical statistical, neural network, and foundation models. Journal of Systems and Software 240, 112937. doi:10.1016/j.jss.2026.112937.

Fifty, C., Amid, E., Zhao, Z., Yu, T., Anil, R., Finn, C., 2021. Efficiently identifying task groupings for multi-task learning, in: Advances in Neural Information Processing Systems, Curran Associates, Inc.. pp. 27503–27516. URL: https://proceedings.neurips.cc/paper/2021/hash/e77910ebb93b511588557806310f78f1-Abstract.html.

Gao, M., Yu, Z., Luo, Y., Ma, L., Sun, Y., Fang, Y., Gao, X., 2026a. Macro-micro collaborative learning for logical data center microservice indicators forecasting, in: Proceedings of the ACM Web Conference 2026, ACM. pp. 5010–5020. doi:10.1145/3774904.3792125.

Gao, X., Tang, X., Yao, C., Wu, S., Yuan, G., Zhou, W., Li, F., Chen, G., 2026b. OMCR: An online multivariate forecaster for cloud resource management. IEEE Transactions on Knowledge and Data Engineering 38, 532–545. doi:10.1109/TKDE.2025.3619097.

Hyndman, R.J., Athanasopoulos, G., 2021. Forecasting: Principles and Practice. 3rd ed., OTexts, Melbourne, Australia. URL: https://otexts.com/fpp3/. accessed 2026-10-16.

Hyndman, R.J., Koehler, A.B., 2006. Another look at measures of forecast accuracy. International Journal of Forecasting 22, 679–688. doi:10.1016/j.ijforecast.2006.03.001.

Kohne, A., Pasternak, D., Nagel, L., Spinczyk, O., 2016. Evaluation of SLA-based decision strategies for VM scheduling in cloud data centers, in: Proceedings of the 3rd Workshop on CrossCloud Infrastructures & Platforms, ACM. pp. 6:1–6:5. doi:10.1145/2904111.2904113.

Kohne, A., Spohr, M., Nagel, L., Spinczyk, O., 2014. FederatedCloudSim: A SLA-aware federated cloud simulation framework, in: Proceedings of the 2nd International Workshop on CrossCloud Systems, ACM. pp. 3:1–3:5. doi:10.1145/2676662.2676674.

Liu, Y., Hu, T., Zhang, H., Wu, H., Wang, S., Ma, L., Long, M., 2024. iTransformer: Inverted transformers are effective for time series forecasting, in: The Twelfth International Conference on Learning Representations. URL: https://openreview.net/forum?id=JePfAI8fah.

Meliani, A.E., Arora, S., Ksentini, A., Knopp, R., 2025. TimeTrack: A dataset for exploring temporal patterns and predictive insights into OpenAirInterface (OAI) CI/CD cluster, in: ICC 2025 - IEEE International Conference on Communications, IEEE. pp. 1978–1983. doi:10.1109/ICC52391.2025.11161298.

Montero-Manso, P., Hyndman, R.J., 2021. Principles and algorithms for forecasting groups of time series: Locality and globality. International Journal of Forecasting 37, 1632–1653. doi:10.1016/j.ijforecast.2021.03.004.

Nie, Y., Nguyen, N.H., Sinthong, P., Kalagnanam, J., 2023. A time series is worth 64 words: Long-term forecasting with transformers, in: The Eleventh International Conference on Learning Representations. URL: https://openreview.net/forum?id=Jbdc0vTOcol.

Nikentari, N., Wei, H.L., 2022. Multi-task learning for time series forecasting using NARMAX-LSTM, in: 2022 27th International Conference on Automation and Computing (ICAC), IEEE. pp. 1–6. doi:10.1109/ICAC55051.2022.9911071.

Nochumsohn, L., Zisling, H., Azencot, O., 2025. A multi-task learning approach to linear multivariate forecasting, in: Proceedings of The 28th International Conference on Artificial Intelligence and Statistics, PMLR. pp. 2638–2646. URL: https://proceedings.mlr.press/v258/nochumsohn25a.html.

Qu, C., Calheiros, R.N., Buyya, R., 2018. Auto-scaling web applications in clouds: A taxonomy and survey. ACM Computing Surveys 51, 73:1–73:33. doi:10.1145/3148149.

Rossi, A., Visentin, A., Carraro, D., Prestwich, S., Brown, K.N., 2025. Forecasting workload in cloud computing: towards uncertainty-aware predictions and transfer learning. Cluster Computing 28, 258. doi:10.1007/s10586-024-04933-2.

Salinas, D., Flunkert, V., Gasthaus, J., Januschowski, T., 2020. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. International Journal of Forecasting 36, 1181–1191. doi:10.1016/j.ijforecast.2019.07.001.

Sener, O., Koltun, V., 2018. Multi-task learning as multi-objective optimization, in: Advances in Neural Information Processing Systems, Curran Associates, Inc.. pp. 525–536. URL: https://proceedings.neurips.cc/paper/2018/hash/432aca3a1e345e339f35a30c8f65edce-Abstract.html.

Shen, S., van Beek, V., Iosup, A., 2015. Statistical characterization of business-critical workloads hosted in cloud datacenters, in: 2015 15th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing, IEEE. pp. 465–474. doi:10.1109/CCGrid.2015.60.

Standley, T., Zamir, A., Chen, D., Guibas, L., Malik, J., Savarese, S., 2020. Which tasks should be learned together in multi-task learning?, in: Proceedings of the 37th International Conference on Machine Learning, PMLR. pp. 9120–9132. URL: https://proceedings.mlr.press/v119/standley20a.html.

Syu, Y., Kuo, J.Y., Fanjiang, Y.Y., 2017. Time series forecasting for dynamic quality of web services: An empirical study. Journal of Systems and Software 134, 279–303. doi:10.1016/j.jss.2017.09.011.

Taherizadeh, S., Jones, A.C., Taylor, I., Zhao, Z., Stankovski, V., 2018. Monitoring self-adaptive applications within edge computing frameworks: A state-of-the-art review. Journal of Systems and Software 136, 19–38. doi:10.1016/j.jss.2017.10.033.

Tran, N., Nguyen, T., Nguyen, B.M., Nguyen, G., 2018. A multivariate fuzzy time series resource forecast model for clouds using LSTM and data correlation analysis. Procedia Computer Science 126, 636–645. doi:10.1016/j.procs.2018.07.298.

Tripathi, S., Priyadarshni, Misra, R., Singh, T.N., 2025. Multilayer multivariate forecasting network for precise resource utilization prediction in edge data centers. Future Generation Computer Systems 166, 107692. doi:10.1016/j.future.2024.107692.

Ullah, F., Bilal, M., Yoon, S.K., 2023. Intelligent time-series forecasting framework for non-linear dynamic workload and resource prediction in cloud. Computer Networks 225, 109653. doi:10.1016/j.comnet.2023.109653.

Xu, M., Song, C., Wu, H., Gill, S.S., Ye, K., Xu, C., 2022. esDNN: Deep neural network based multivariate workload prediction in cloud computing environments. ACM Transactions on Internet Technology 22, 75:1–75:24. doi:10.1145/3524114.

Yu, J., Liu, X., Luo, C., Huang, J., Zhou, R., Liu, Y., Hu, J., Chen, J., Zhang, K., Zhang, D., Shen, Y., Adhikarla, E., Dai, Y., Zhang, K., Kong, Z., Ye, W., Yin, Y., Namboodiri, V., Davison, B., Moore, J., Chen, Y., 2025. Multitask learning 1997–2024: Part I fundamentals. Harvard Data Science Review 7. URL: https://hdsr.mitpress.mit.edu/pub/7fcc3jhv/release/2, doi:10.1162/99608f92.462f48a2.

Yu, T., Kumar, S., Gupta, A., Levine, S., Hausman, K., Finn, C., 2020. Gradient surgery for multi-task learning, in: Advances in Neural Information Processing Systems, Curran Associates, Inc.. pp. 5824–5836. URL: https://proceedings.neurips.cc/paper/2020/hash/3fe78a8acf5fda99de95303940a2420c-Abstract.html.

Zhang, B., Yang, G., Yu, F., Pan, Q., 2025. SWT-CLSTM: A hybrid model for cloud workload prediction combining smooth wavelet transform and contrastive learning. Journal of King Saud University Computer and Information Sciences 37, 320. doi:10.1007/s44443-025-00316-8.